Assignment 3 — LSTM Sequence Prediction: Learning the Alphabet

DATA-MSML 612 — Deep Learning
University of Maryland

Shrikanth Vilvadrinath

March 2026

Contents

1 Introduction **2**

2 Dataset and Preprocessing **2**
2.1 Character-to-Integer Mapping . . . 2
2.2 Input Reshaping . . . 2
2.3 Normalization . . . 2
2.4 One-Hot Encoding . . . 3

3 Model Architecture **3**
3.1 Baseline Model . . . 3
3.2 Stacked LSTM Architecture (Step 10) . . . 3

4 Results **4**
4.1 Baseline Performance (Steps 7–9) . . . 4
4.2 Depth Experiments (Step 10) . . . 4
4.3 Hyperparameter Experiments (Step 11) . . . 6
4.3.1 Sweep A: Learning Rate . . . 6
4.3.2 Sweep B: Hidden Size . . . 7
4.3.3 Sweep C: Number of Layers (Hyperparameter Selection) . . . 7

5 Analysis **8**
5.1 Nature of the Task: Training-Set Memorisation . . . 8
5.2 Single-Timestep Input and LSTM Recurrent Memory . . . 9
5.3 Depth Trade-offs . . . 9
5.4 Hyperparameter Interactions . . . 9

6 Conclusion **9**

1 Introduction

Recurrent Neural Networks (RNNs) are designed to process sequential data by maintaining a hidden state across time steps. However, standard RNNs suffer from the **vanishing and exploding gradient problem** during backpropagation through time, making it difficult to learn long-range dependencies.

Long Short-Term Memory (LSTM) networks [1] address this limitation through a gating mechanism with three learned gates:

- **Forget gate:** Decides what fraction of the prior cell state to retain.
- **Input gate:** Controls what new information is written to the cell state.
- **Output gate:** Determines what portion of the cell state is exposed as the hidden state output.

This assignment implements an LSTM model for a simple sequence prediction task: given a letter of the alphabet, predict the next letter. The task is framed as a 26-class classification problem with 25 input-output pairs (A→B, B→C, . . . , Y→Z). The target accuracy is **80**%.

Framework: TensorFlow 2.10 / Keras. **Seed:** 42 (fully reproducible).

2 Dataset and Preprocessing

The dataset is the English alphabet: 26 characters (A–Z), producing 25 sequential input-output pairs.

2.1 Character-to-Integer Mapping

Each character is mapped to an integer: A=0, B=1, . . . , Z=25, and the reverse mapping is constructed for decoding predictions.

2.2 Input Reshaping

The input array `X` is reshaped to `[25, 1, 1]` corresponding to `[samples, timesteps, features]`:

- **25 samples:** One for each letter pair.
- **1 timestep:** Single-character context (one letter at a time).
- **1 feature:** The normalized integer encoding of the character.

2.3 Normalization

Input values are normalized to $[0.0, 1.0]$ by dividing by 25 (the number of intervals between 26 characters):

$$x_{\text{norm}} = \frac{x_{\text{int}}}{25}$$

2.4 One-Hot Encoding

The output variable is one-hot encoded into 26-dimensional vectors using `to_categorical`. For example, B (class 1) becomes $[0, 1, 0, 0, \ldots, 0]$.

3 Model Architecture

3.1 Baseline Model

Listing 1: Baseline LSTM architecture

```
LSTM(32, input_shape=(1, 1))     -> output: (batch, 32)
Dense(26, activation='softmax')  -> output: (batch, 26)
# Total trainable parameters: 5,210
```

Parameter breakdown:

- `LSTM(32)`: Input dimension 1, hidden dimension 32. Each LSTM gate uses weight matrices of size $(d_{\text{in}} + d_h) \times d_h$ plus a bias vector. Total: $4 \times (1 \times 32 + 32 \times 32 + 32) = 4{,}352$ parameters.
- `Dense(26, softmax)`: Projection from 32-dimensional hidden state to 26 class probabilities. Total: $32 \times 26 + 26 = 858$ parameters.
- **Grand total:** $4{,}352 + 858 = \mathbf{5{,}210}$ trainable parameters.

Compile settings:

- Optimizer: `Adam(learning_rate=0.01)` – higher than the default 0.001 to accelerate convergence on this tiny dataset.
- Loss: `categorical_crossentropy` – standard for multi-class classification.
- Training: 500 epochs, `batch_size=1` (stochastic updates over 25 samples).

3.2 Stacked LSTM Architecture (Step 10)

For depth experiments, LSTM layers are stacked with `return_sequences=True` on all but the final LSTM layer, so each intermediate layer passes its full output sequence to the next:

Listing 2: Stacked LSTM builder (depth-variable)

```
def build_lstm_model(num_layers, hidden_units, learning_rate):
    model = Sequential()
    for i in range(num_layers):
        return_seq = (i < num_layers - 1)
        model.add(LSTM(hidden_units, return_sequences=return_seq,
                       input_shape=(1, 1) if i == 0 else None))
    model.add(Dense(26, activation='softmax'))
    model.compile(loss='categorical_crossentropy',
                  optimizer=Adam(learning_rate=learning_rate),
                  metrics=['accuracy'])
    return model
```

4 Results

4.1 Baseline Performance (Steps 7–9)

Table 1 shows the baseline single-layer LSTM performance after 500 epochs of training.

Table 1: Baseline LSTM model performance (SEED=42, 500 epochs)

Metric	**Value**
Loss	0.6317
Accuracy	100.00%
Correct	25/25 letters
Target	80%
Result	**PASS** (+20.00%)

Table 2 shows the complete Step 9 prediction evidence: all 25 input letters are mapped to the correct successor with 100% accuracy.

Table 2: Step 9: Full prediction table — all 25/25 correct (✓)

In→Out		**In→Out**		**In→Out**		**In→Out**		**In→Out**	
A→B	✓	F→G	✓	K→L	✓	P→Q	✓	U→V	✓
B→C	✓	G→H	✓	L→M	✓	Q→R	✓	V→W	✓
C→D	✓	H→I	✓	M→N	✓	R→S	✓	W→X	✓
D→E	✓	I→J	✓	N→O	✓	S→T	✓	X→Y	✓
E→F	✓	J→K	✓	O→P	✓	T→U	✓	Y→Z	✓

4.2 Depth Experiments (Step 10)

Per the assignment specification, Step 10 begins with a **2-layer stacked LSTM** as the primary configuration and varies the number of layers. A 1-layer model is included as a reference baseline. The focus of this step is *convergence behaviour*: accuracy and loss trajectories over 500 training epochs.

Table 3 summarises the final-epoch results.

Table 3: Effect of LSTM depth on convergence (hidden=32, lr=0.01, 500 epochs)

Depth	**Role**	**Accuracy (%)**	**Loss**
1	Reference baseline	100.00	0.6317
2	Spec anchor (stacked)	100.00	0.0378
3	Extended depth	100.00	0.0822
4	Extended depth	40.00	1.8950

Depths 1–3 all converge to 100% accuracy within 500 epochs. The 2-layer stacked LSTM achieves the lowest final loss (0.0378), indicating faster and more complete convergence

than the 1-layer baseline. The 4-layer model fails to converge within the 500-epoch budget: with only 25 training samples, the additional parameters and gradient path length require substantially more training iterations.

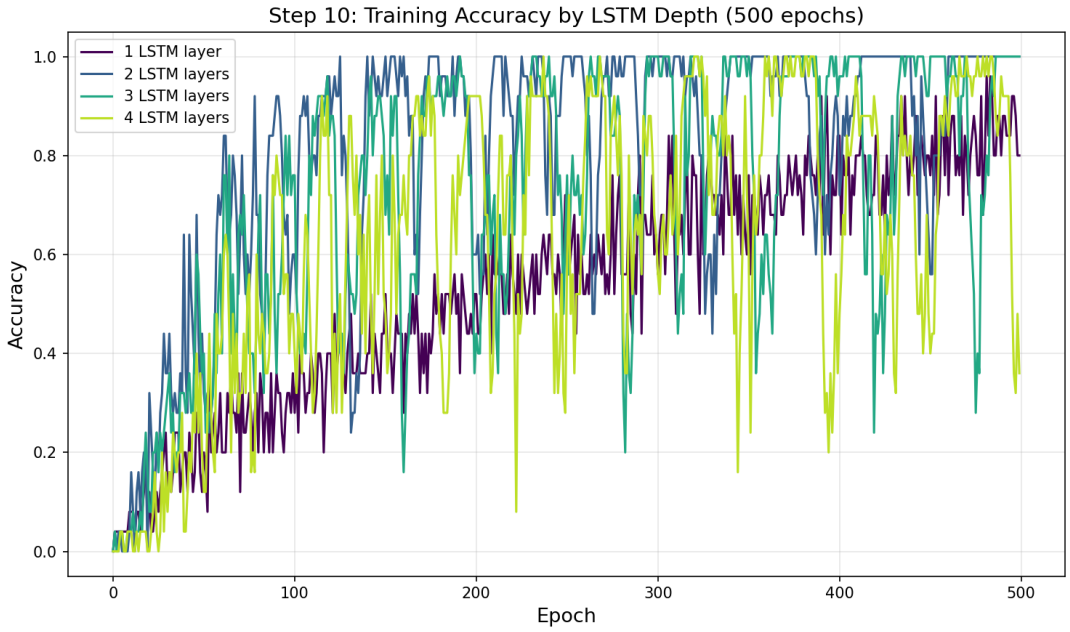

Figure 1: Training accuracy curves for 1–4 LSTM layers over 500 epochs. Depths 1–3 converge to 100%; depth 4 fails to converge within the epoch budget.

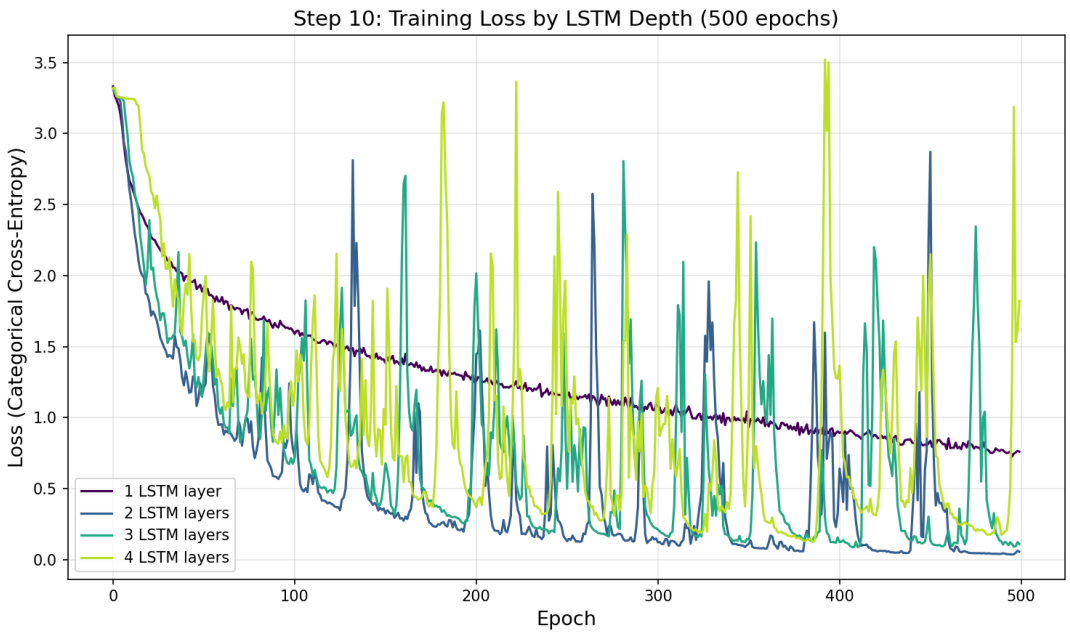

Figure 2: Training loss curves for 1–4 LSTM layers over 500 epochs. The 2-layer stacked LSTM achieves the lowest final loss.

4.3 Hyperparameter Experiments (Step 11)

Three independent sweeps were conducted, each varying one parameter while holding others at the baseline (hidden=32, layers=1, lr=0.01, 500 epochs, batch=1).

4.3.1 Sweep A: Learning Rate

Table 4: Effect of learning rate on final accuracy (hidden=32, layers=1, 500 epochs)

Learning Rate	Accuracy (%)	Loss
0.0001	8.00	2.9093
0.001	84.00	1.6479
0.005	96.00	0.9989
0.01	**100.00**	**0.6317**
0.05	92.00	0.8086
0.1	96.00	0.5517

Optimal: lr = 0.01. Very small learning rates (0.0001) produce severe underfitting within 500 epochs (8% accuracy). The rate 0.001 reaches 84% but leaves headroom. Rates from 0.005–0.1 all exceed 80%, but 0.01 uniquely reaches 100%. Larger rates (0.05, 0.1) converge faster initially but show slightly less complete memorisation at epoch 500, suggesting mild oscillation near the optimum.

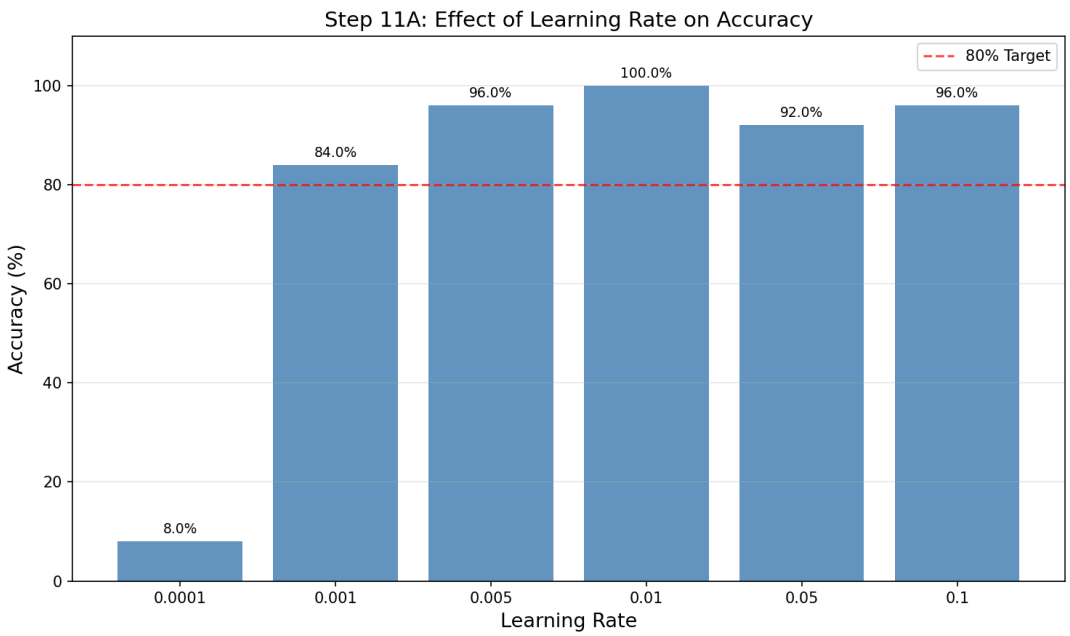

Figure 3: Effect of learning rate on final accuracy after 500 epochs. The red dashed line marks the 80% target. lr=0.01 achieves 100%.

4.3.2 Sweep B: Hidden Size

Table 5: Effect of hidden size on final accuracy (layers=1, lr=0.01, 500 epochs)

Hidden Units	Accuracy (%)	Loss
4	96.00	1.1923
8	96.00	0.8952
16	96.00	0.7905
32	**100.00**	**0.6317**
64	100.00	0.4624
128	100.00	0.3762

Optimal: hidden = 32 (smallest size achieving 100%; selected by parsimony). Sizes 4–16 plateau at 96% – they lack sufficient representational capacity to distinguish all 25 mappings. Sizes 32, 64, and 128 all achieve 100%, with diminishing loss improvement beyond 32. The baseline of 32 is chosen as the optimal value per the Occam's razor principle: minimum necessary capacity.

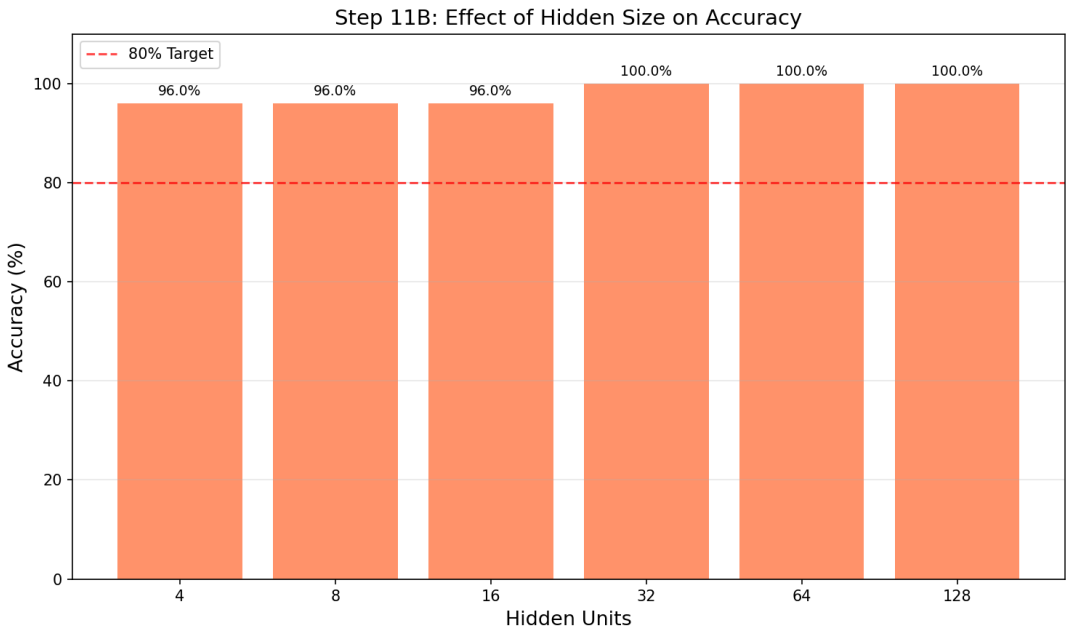

Figure 4: Effect of hidden layer size on final accuracy. Sizes ≥ 32 all achieve 100%; smaller sizes plateau at 96%.

4.3.3 Sweep C: Number of Layers (Hyperparameter Selection)

Note: Step 10 examined convergence trajectories (epoch-by-epoch curves) for architectural analysis. This sweep reports the single final-accuracy figure at epoch 500 for hyperparameter selection. The overlapping depth range serves a distinct analytical purpose.

Table 6: Effect of number of layers on final accuracy (hidden=32, lr=0.01, 500 epochs)

Layers	Accuracy (%)	Loss
1	**100.00**	**0.6317**
2	100.00	0.0378
3	100.00	0.0822
4	40.00	1.8950
5	100.00	0.1530

Optimal: layers = 1 (selected by parsimony). Layers 1, 2, 3, and 5 all achieve 100% accuracy. Layer 4 fails to converge within 500 epochs, consistent with the Step 10 finding. Since the simplest model achieves perfect accuracy on this 25-sample task, adding layers introduces unnecessary parameters and convergence risk without accuracy benefit.

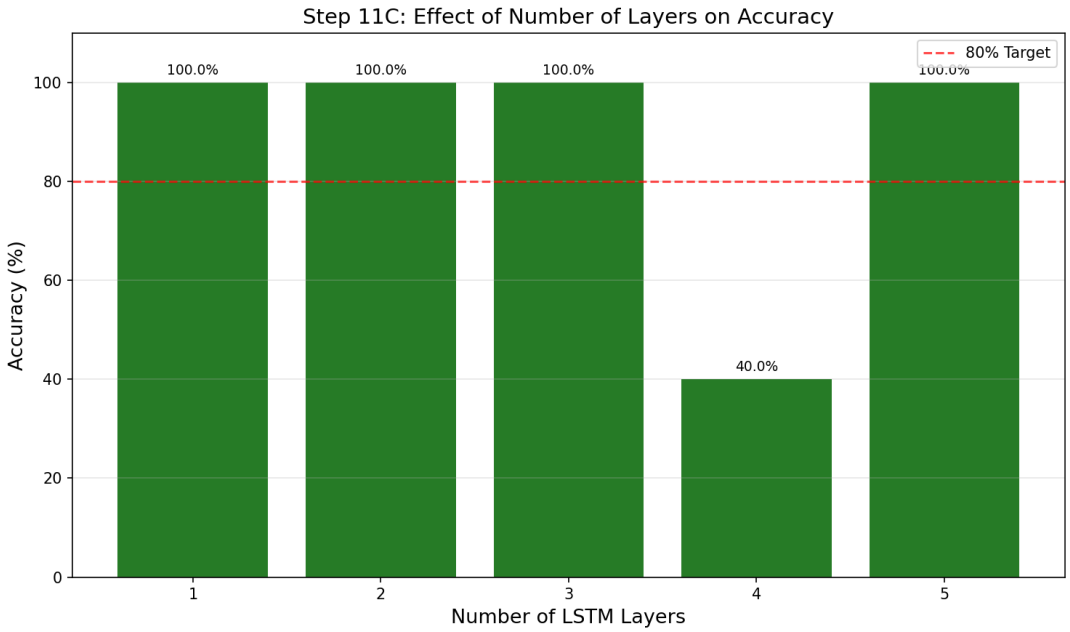

Figure 5: Effect of number of LSTM layers on final accuracy. Most configurations achieve 100%; depth 4 fails to converge within 500 epochs.

5 Analysis

5.1 Nature of the Task: Training-Set Memorisation

This is a **training-set memorisation task**, not a generalisation benchmark. The model is evaluated on the same 25 samples it was trained on, with no held-out test set. Accuracy therefore reflects the model's ability to memorise 25 fixed input-output pairs, not to generalise to unseen sequences. This is a deliberate design choice per the assignment specification and is the standard framing for this pedagogical exercise.

5.2 Single-Timestep Input and LSTM Recurrent Memory

With `timesteps=1`, each of the 25 samples is processed in a single LSTM forward pass. Because there is only one time step per sample, the recurrent connections carry no information *within* a sample: the hidden state computed at step $t = 1$ is the final hidden state, and there are no prior time steps to inform it. The task collapses into a **one-step mapping problem** from a single normalised input scalar to one of 26 output classes. The LSTM's gating mechanism — designed to selectively retain information *across multiple time steps* — is not meaningfully exercised here.

The LSTM succeeds despite this limitation because:

1. The 32-dimensional hidden projection provides sufficient capacity to encode 25 distinct input-output mappings.
2. Adam at lr=0.01 converges efficiently on this small problem.
3. 500 epochs provide ample gradient updates for the 25-sample dataset.

5.3 Depth Trade-offs

Adding LSTM layers increases both parameter count and gradient path length. On a 25-sample task, this introduces two risks:

- **Over-parameterisation:** More weights than are needed to solve a trivial mapping task.
- **Slower convergence:** Gradients must flow through more LSTM gates, and the fixed 500-epoch budget may be insufficient. This is evident in the depth-4 failure (40% accuracy).

The step-10 convergence curves confirm that the 2-layer stacked LSTM converges faster (lower loss at epoch 500) than the 1-layer baseline, while the 4-layer model diverges – consistent with the over-parameterisation hypothesis.

5.4 Hyperparameter Interactions

- **Learning rate** is the most impactful hyperparameter on this task. The range 0.005–0.1 all exceed 80%, but only 0.01 achieves 100% at epoch 500. The effect is strong: lr=0.0001 achieves only 8%, confirming that the 500-epoch budget is the binding constraint at very small learning rates.
- **Hidden size** shows a capacity threshold. Below 32 units, the model lacks sufficient representational capacity for 25 distinct class mappings; at 32 and above, the task is within capacity and all sizes perform equally.
- **Depth** interacts with convergence budget: deeper models need more epochs or a higher learning rate to compensate for increased gradient path length.

6 Conclusion

The LSTM model successfully learns the alphabet sequence prediction task, achieving **100% accuracy** with the baseline configuration — well above the 80% target. Key

findings:

1. The **baseline model** (1 LSTM layer, 32 hidden units, lr=0.01) correctly predicts all 25 next-letter mappings.
2. The **2-layer stacked LSTM** (Step 10 spec anchor) also achieves 100% accuracy with a lower final loss (0.0378 vs. 0.6317), indicating more complete convergence.
3. **Depth 4 fails to converge** within 500 epochs, confirming that deeper networks require a larger training budget on this small dataset.
4. **lr=0.01** is the optimal learning rate; lr=0.0001 produces severe underfitting (8%) within the fixed epoch budget.
5. **Hidden size 32** is the minimum capacity threshold for 100% accuracy; smaller sizes plateau at 96%.
6. The **optimal configuration** is lr=0.01, hidden=32, layers=1 — the simplest model that achieves perfect accuracy.
7. This task is a **one-step mapping problem**: with a single input timestep, the LSTM's recurrent memory mechanism is not exercised. Accuracy reflects memorisation capacity, not sequential generalisation.

References

[1] Hochreiter, S. and Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8), pp. 1735–1780.

[2] Chollet, F. (2018). *Deep Learning with Python.* Manning Publications.

[3] Keras documentation — LSTM layer. https://keras.io/api/layers/recurrent_layers/lstm/

Code: `Assignment 3/code/assignment3.py` *Plots:* `Assignment 3/output/` *Result tables:* `output/baseline_results.txt`, `step10_depth_results.csv`, `step11_*.csv`